

Mounted at /content/drive

```

/usr/local/lib/python3.13/dist-packages/numpy/_core/fromnumeric.py:57: FutureWarning: 'DataFrame.swapaxes' is deprecated and will be
    return bound(*args, **kwargs)
LSWMD_01.pkl 저장 완료: (40573, 6)
LSWMD_02.pkl 저장 완료: (40573, 6)
LSWMD_03.pkl 저장 완료: (40573, 6)
LSWMD_04.pkl 저장 완료: (40573, 6)
LSWMD_05.pkl 저장 완료: (40573, 6)
LSWMD_06.pkl 저장 완료: (40573, 6)
LSWMD_07.pkl 저장 완료: (40573, 6)

```

```

LSWMD_08.pkl 저장 완료: (40573, 6)
LSWMD_09.pkl 저장 완료: (40573, 6)
LSWMD_10.pkl 저장 완료: (40573, 6)
LSWMD_11.pkl 저장 완료: (40573, 6)
LSWMD_12.pkl 저장 완료: (40573, 6)
LSWMD_13.pkl 저장 완료: (40573, 6)
LSWMD_14.pkl 저장 완료: (40573, 6)
LSWMD_15.pkl 저장 완료: (40573, 6)
LSWMD_16.pkl 저장 완료: (40573, 6)
LSWMD_17.pkl 저장 완료: (40573, 6)
LSWMD_18.pkl 저장 완료: (40572, 6)
LSWMD_19.pkl 저장 완료: (40572, 6)
LSWMD_20.pkl 저장 완료: (40572, 6)

```

```

import gc
import pandas as pd

# 1. 기존 큰 DataFrame 제거
del df
gc.collect()

# 2. 분할 파일 읽기
df = pd.read_pickle(
    '/content/drive/MyDrive/_for_exercise/WM811K_split/LSWMD_01.pkl'
)

# 3. 확인
print(df.shape)
df.head()

```

```

(40573, 6)

```

		waferMap	dieSize	lotName	waferIndex	trianTestLabel	failureType
0	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...		1683.0	lot1	1.0	[[Training]]	[[none]]
1	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...		1683.0	lot1	2.0	[[Training]]	[[none]]
2	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...		1683.0	lot1	3.0	[[Training]]	[[none]]
3	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...		1683.0	lot1	4.0	[[Training]]	[[none]]
4	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...		1683.0	lot1	5.0	[[Training]]	[[none]]

▾ 본격 탐색 작업

```

root_path = '/content/drive/MyDrive/_for_exercise/WM811K_split/'

```

```

import pandas as pd

path = '/content/drive/MyDrive/_for_exercise/WM811K_split/LSWMD_01.pkl'

df = pd.read_pickle(path)

print(df.shape)
print(df.columns)
df.head()
df.sample(10)

```

```
(40573, 6)
Index(['waferMap', 'dieSize', 'lotName', 'waferIndex', 'trianTestLabel',
      'failureType'],
      dtype='object')
```

	waferMap	dieSize	lotName	waferIndex	trianTestLabel	failureType
528	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 1, 2, 1,...	533.0	lot22	4.0	[[Training]]	[[none]]
7309	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2,...	1091.0	lot490	6.0	[[Training]]	[[none]]
38754	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...	776.0	lot3494	6.0	[]	[]
19119	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...	1349.0	lot2422	11.0	[]	[]
187	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2,...	533.0	lot8	13.0	[[Training]]	[[none]]
3881	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,...	533.0	lot157	7.0	[[Training]]	[[none]]
445	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 2, 1, 1,...	533.0	lot18	21.0	[]	[]
11707	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...	2995.0	lot2007	17.0	[]	[]
27119	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 1, 1, 2, 1,...	674.0	lot2892	22.0	[]	[]
35330	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...	1253.0	lot3278	1.0	[]	[]

```
# colum 이름에 오타 수정
cols = [
    'waferMap',
    'dieSize',
    'lotName',
    'waferIndex',
    'trainTestLabel',
    'failureType'
]

df.columns = cols
df.head()
```

	waferMap	dieSize	lotName	waferIndex	trainTestLabel	failureType
0	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...	1683.0	lot1	1.0	[[Training]]	[[none]]
1	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...	1683.0	lot1	2.0	[[Training]]	[[none]]
2	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...	1683.0	lot1	3.0	[[Training]]	[[none]]
3	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...	1683.0	lot1	4.0	[[Training]]	[[none]]
4	[[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...	1683.0	lot1	5.0	[[Training]]	[[none]]

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40573 entries, 0 to 40572
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   waferMap        40573 non-null  object
1   dieSize         40573 non-null  float64
2   lotName         40573 non-null  object
3   waferIndex      40573 non-null  float64
4   trianTestLabel  40573 non-null  object
5   failureType     40573 non-null  object
dtypes: float64(2), object(4)
memory usage: 1.9+ MB
```

```
print(df.columns)
print()

for col in df.columns:
    print(f'[{col}]')
    print('type:', type(df[col].iloc[0]))
    print('sample:', df[col].iloc[0])
    print('-' * 50)
```

```
Index(['waferMap', 'dieSize', 'lotName', 'waferIndex', 'trianTestLabel',
      'failureType'],
      dtype='object')

[waferMap]
type: <class 'numpy.ndarray'>
```

```
sample: [[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
...
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]
[0 0 0 ... 0 0 0]]
```

```
[dieSize]
type: <class 'numpy.float64'>
sample: 1683.0
```

```
[lotName]
type: <class 'numpy.str_'>
sample: lot1
```

```
[waferIndex]
type: <class 'numpy.float64'>
sample: 1.0
```

```
[trianTestLabel]
type: <class 'numpy.ndarray'>
sample: [['Training']]
```

```
[failureType]
type: <class 'numpy.ndarray'>
sample: [['none']]
```

▽ 의사결정에 필요한 정보

```
sample = df['waferMap'].iloc[0]
```

```
print(type(sample))
print(sample.shape)
print(sample.dtype)
print(sample)
```

```
<class 'numpy.ndarray'>
(45, 48)
uint8
[[0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]]
```

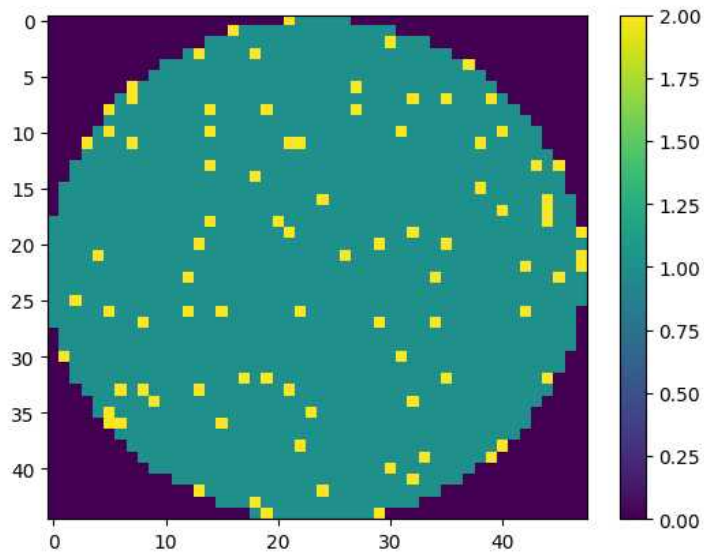
```
import numpy as np
```

```
np.unique(sample)
```

```
array([0, 1, 2], dtype=uint8)
```

```
import matplotlib.pyplot as plt
```

```
plt.imshow(sample)
plt.colorbar()
plt.show()
```



```
print(df['dieSize'].head())
print("-"*50)
print(df['dieSize'].describe())
```

```
0    1683.0
1    1683.0
2    1683.0
3    1683.0
4    1683.0
Name: dieSize, dtype: float64
```

```
-----
count    40573.000000
mean      1440.941094
std       1832.744961
min         12.000000
25%        710.000000
50%        741.000000
75%       1075.000000
max       21338.000000
Name: dieSize, dtype: float64
```

```
print(df['lotName'].head())
print(df['lotName'].nunique())
```

```
0    lot1
1    lot1
2    lot1
3    lot1
4    lot1
Name: lotName, dtype: object
2487
```

```
print(df['waferIndex'].head())

df['waferIndex'].value_counts()
```

```
0    1.0
1    2.0
2    3.0
3    4.0
4    5.0
```

Name: waferIndex, dtype: float64

	count
waferIndex	

4.0	1687
6.0	1686
5.0	1680
3.0	1680
7.0	1673
2.0	1669
9.0	1660
1.0	1648
8.0	1645
10.0	1642
12.0	1642
11.0	1636
13.0	1630
14.0	1627
15.0	1601
21.0	1592
18.0	1590
17.0	1589
16.0	1583
22.0	1578
24.0	1577
20.0	1574
23.0	1565
19.0	1560
25.0	1559

dtype: int64

```
train_label = df['trainTestLabel'].apply(
    lambda x: x[0][0] if len(x) > 0 else None
)
```

```
train_label.value_counts(dropna=False)
```

	count
trainTestLabel	
None	32180
Training	8393

dtype: int64

```
# 오래 걸림. 평탄화 필요
# df['trainTestLabel'].value_counts(dropna=False)
```

```
failure_type = df['failureType'].apply(
    lambda x: x[0][0] if len(x) > 0 else None
)

failure_type.value_counts(dropna=False)
```

	count
failureType	
None	32180
none	6383
Edge-Ring	1370
Edge-Loc	228
Random	165
Loc	130
Scratch	54
Donut	30
Center	25
Near-full	8

dtype: int64

▽ 평탄화

```
# 빈 배열이 섞여 있을 수 있어서, 조금 더 안전하게 이렇게 쓰는 걸 추천.
def extract_label(x):
    try:
        return x[0][0]
    except:
        return None
```

```
# 그럼 이렇게 사용 가능
train_label = df['trainTestLabel'].apply(extract_label)
failure_type = df['failureType'].apply(extract_label)
```

```
print(train_label.value_counts(dropna=False))
print(failure_type.value_counts(dropna=False))
```

```
trainTestLabel
None      32180
Training   8393
Name: count, dtype: int64
failureType
None      32180
none      6383
Edge-Ring  1370
Edge-Loc   228
Random     165
Loc        130
Scratch     54
Donut       30
Center      25
Near-full   8
Name: count, dtype: int64
```

```
# 평탄화 한 값을 아예 새 컬럼으로 붙이기
df['trainLabel_clean'] = df['trainTestLabel'].apply(extract_label)
df['failureType_clean'] = df['failureType'].apply(extract_label)

df['failureType_clean'].value_counts()
```


	count
waferShape	
(33, 29)	10366
(26, 26)	7655
(30, 34)	3783
(40, 40)	1867
(32, 29)	1434
(34, 34)	1365
(38, 36)	1134
(34, 35)	1098
(39, 31)	1066
(87, 89)	869
(54, 71)	869
(118, 84)	789
(113, 71)	773
(41, 41)	623
(25, 27)	482
(89, 76)	382
(47, 42)	325
(25, 28)	309
(26, 30)	306
(29, 30)	275

dtype: int64

```
# 웨이퍼맵 크기 전체 종류 수
df['waferShape'].nunique()
```

162

2. dieSize와 실제 waferMap 크기의 관계

```
# 2. dieSize와 실제 waferMap 크기의 관계

df[['dieSize', 'waferShape']].head(20)
```

	dieSize	waferShape
0	1683.0	(45, 48)
1	1683.0	(45, 48)
2	1683.0	(45, 48)
3	1683.0	(45, 48)
4	1683.0	(45, 48)
5	1683.0	(45, 48)
6	1683.0	(45, 48)
7	1683.0	(45, 48)
8	1683.0	(45, 48)
9	1683.0	(45, 48)
10	1683.0	(45, 48)
11	1683.0	(45, 48)
12	1683.0	(45, 48)
13	1683.0	(45, 48)
14	1683.0	(45, 48)
15	1683.0	(45, 48)
16	1683.0	(45, 48)
17	1683.0	(45, 48)
18	1683.0	(45, 48)
19	1683.0	(45, 48)

```
df[['dieSize', 'waferShape']].sample(20)
```

	dieSize	waferShape
37546	5342.0	(89, 76)
21159	1075.0	(38, 36)
16942	6295.0	(113, 71)
28904	741.0	(33, 29)
29392	741.0	(33, 29)
17360	741.0	(33, 29)
38705	776.0	(30, 34)
31674	4535.0	(72, 80)
7466	1724.0	(50, 43)
25225	2858.0	(54, 65)
16237	741.0	(33, 29)
30400	741.0	(33, 29)
37633	5342.0	(89, 76)
33189	741.0	(33, 29)
4653	533.0	(26, 26)
35995	710.0	(32, 29)
38878	776.0	(30, 34)
4208	533.0	(26, 26)
13366	924.0	(34, 35)
3277	533.0	(26, 26)

```
# 같은 dieSize인데 shape가 같은지 다른지
df.groupby('dieSize')['waferShape'].nunique().sort_values(ascending=False).head(20)
```

[숨겨진 출력 표시](#)

```
sample = df['waferMap'].iloc[0]

print('dieSize:', df['dieSize'].iloc[0])
print('0이 아닌 픽셀 수:', (sample != 0).sum())
```

```
dieSize: 1683.0
0이 아닌 픽셀 수: 1683
```

3. 클래스별 웨이퍼 몇 장 직접 보기

3. 클래스별 웨이퍼 몇 장 직접 보기

```
# 라벨이 있는 데이터만 선택
labeled = df[df['failureType_clean'].notna()]
```

```
import matplotlib.pyplot as plt

classes = labeled['failureType_clean'].unique()

plt.figure(figsize=(15, 8))

for i, cls in enumerate(classes):
    sample = labeled[labeled['failureType_clean'] == cls].iloc[0]

    plt.subplot(3, 3, i + 1)
    plt.imshow(sample['waferMap'])
    plt.title(cls)
    plt.axis('off')

plt.tight_layout()
plt.show()
```

[숨겨진 출력 표시](#)

이미지 크기별로 failure class 분포 확인

```
labeled.head()
```

[숨겨진 출력 표시](#)

```
'waferShape' in df.columns
```

```
True
```

```
labeled.shape
```

```
(8393, 9)
```

```
labeled.groupby('waferShape').size().head()
```

[숨겨진 출력 표시](#)

```
labeled.groupby('waferShape').size().sort_values(ascending=False).head(10)
```

[숨겨진 출력 표시](#)

```
labeled.groupby('failureType_clean').size()
```

[숨겨진 출력 표시](#)

```
labeled.groupby(['waferShape', 'failureType_clean']).size()
```

[숨겨진 출력 표시](#)

```
grouped = labeled.groupby(['waferShape', 'failureType_clean']).size()
```

```
grouped.unstack(fill_value=0)
```

[숨겨진 출력 표시](#)

```
shape_class = labeled.groupby(['waferShape', 'failureType_clean']).size().unstack(fill_value=0)
```

```
shape_class.head(20)
```

[숨겨진 출력 표시](#)

```
shape_class.loc[
    [(33,29), (26,26), (30,34), (40,40), (32,29)]
]
```

[숨겨진 출력 표시](#)

```
# 각 shape에 존재하는 failureType 종류 수
class_count_by_shape = shape_class.gt(0).sum(axis=1).sort_values(ascending=False)
```

```
class_count_by_shape.head(20)
```

[숨겨진 출력 표시](#)

```
top_shapes = class_count_by_shape.head(10).index
```

```
shape_class.loc[top_shapes]
```

failureType_clean	Center	Donut	Edge-Loc	Edge-Ring	Loc	Near-full	Random	Scratch	none
waferShape									
(26, 26)	12	0	88	19	42	8	20	21	6073
(33, 29)	0	1	12	38	2	0	2	6	0
(53, 58)	2	0	8	1	3	0	0	1	106
(30, 34)	0	0	29	2	3	0	2	1	0
(37, 55)	0	8	2	0	1	0	0	0	6
(37, 37)	11	1	0	0	7	0	0	0	23
(32, 29)	0	0	3	1	1	0	1	0	0
(34, 34)	0	0	0	9	2	0	0	1	0
(36, 34)	0	0	1	0	9	0	0	0	7
(41, 41)	0	0	1	14	0	0	124	0	0

✓ 전체 데이터셋에서 (26,26) 분포 확인

```
import pandas as pd
import os
import gc

all_counts = {}

for i in range(1, 21):
    path = os.path.join(root_path, f'LSWMD_{i:02d}.pkl')
    part = pd.read_pickle(path)

    # failureType 문자열 꺼내기
    part['failureType_clean'] = part['failureType'].apply(
        lambda x: x[0][0] if len(x) > 0 else None
    )

    # (26, 26)만 선택
    selected = part[
        part['waferMap'].apply(lambda x: x.shape == (26, 26))
    ]

    # 클래스별 개수
    counts = selected['failureType_clean'].value_counts(dropna=False)

    for label, count in counts.items():
```

```
all_counts[label] = all_counts.get(label, 0) + count

del part
gc.collect()

all_counts
```

```
{np.str_('none'): 13489,
 None: 15712,
 np.str_('Edge-Loc'): 296,
 np.str_('Loc'): 297,
 np.str_('Scratch'): 72,
 np.str_('Random'): 74,
 np.str_('Edge-Ring'): 31,
 np.str_('Center'): 90,
 np.str_('Near-full'): 16,
 np.str_('Donut'): 1}
```

```
classes = ['none', 'Edge-Loc', 'Loc', 'Center', 'Scratch']

fig, axes = plt.subplots(len(classes), 5, figsize=(10, 10))

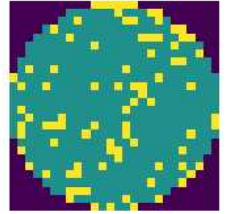
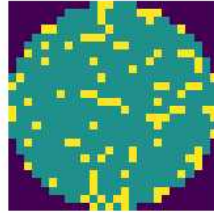
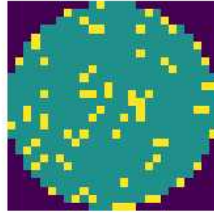
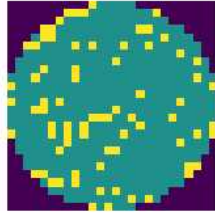
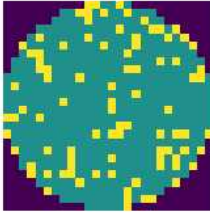
for row, cls in enumerate(classes):
    samples = labeled[
        (labeled['waferShape'] == (26, 26)) &
        (labeled['failureType_clean'] == cls)
    ].head(5)

    for col, (_, sample) in enumerate(samples.iterrows()):
        axes[row, col].imshow(sample['waferMap'])
        axes[row, col].axis('off')

    axes[row, 0].set_title(cls)

plt.tight_layout()
plt.show()
```

none



Edge-Loc

```
# 서브셋 만들기
```

```
# (26,26) 모으기
```

```
import pandas as pd
```

```
import os
```

```
import gc
```

```
folder = '/content/drive/MyDrive/AI비전실무/WM811K_split'
```

```
target_classes = ['none', 'Edge-Loc', 'Center', 'Scratch']
```

```
target_shape = (26, 26)
```

```
collected = []
```

```
for i in range(1, 21):
```

```
    path = os.path.join(root_path, f'LSWMD_{i:02d}.pkl')
```

```
    part = pd.read_pickle(path)
```

```
    # 픽업한 라벨만 가다히 정리
```